Gestion des Processus, Cycle de Vie et Signaux POSIX sous Unix/Linux

Thème principal : Architecture des Systèmes d'Exploitation : Gestion des Tâches, Filiation et Communication Inter-Processus

Synthèse exécutive : Ce cours fondamental de R1.04 aborde l'architecture des systèmes d'exploitation de type Unix/Linux à travers la gestion dynamique des exécutables. Il explicite le passage d'un programme statique (stocké sur disque) à un processus dynamique (actif en mémoire RAM). Pour assurer une simultanéité d'exécution sur le processeur, le noyau met en œuvre le multitâche préemptif et orchestre le cycle de vie des processus via trois états fondamentaux (Élu, Prêt, Bloqué). Le cours détaille l'arborescence généalogique parent-enfant (PID/PPID) initiée par le processus racine (systemd/init, PID 1), le fonctionnement des appels système fork() et exec(), ainsi que la communication asynchrone par signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Attributs, Cycle de Vie et Multitâche

Pour qu'un système d'exploitation exécute des instructions, il doit transformer un programme (code binaire inactif résidant sur le stockage secondaire) en une entité vivante appelée processus (instance chargée en mémoire vive RAM). Dans un système Unix/Linux, la simultanéité des tâches repose sur le multitâche préemptif : le noyau n'attend pas qu'un processus se termine pour passer au suivant, il interrompt autoritairement les tâches à la fin de leur quantum de temps CPU. Le cycle de vie d'un processus est régi par un modèle d'états : lorsqu'il exécute des instructions sur un cœur processeur, il est à l'état 'Élu' (Running). Si son temps alloué expire, l'ordonnanceur le fait repasser à l'état 'Prêt' (Ready). Si le processus demande une lecture sur le disque dur ou attend une saisie clavier, le noyau le bascule à l'état 'Bloqué' (Waiting/Blocked) pour ne pas gaspiller de puissance processeur. Une fois la donnée disponible, le processus n'est jamais réinjecté directement sur le processeur : il retourne dans la file d'attente à l'état 'Prêt'. Chaque processus s'inscrit dans une généalogie stricte. Au démarrage du système, le noyau initialise le processus racine utilisateur systemd (ou init), recevant impérativement le PID 1. Pour exécuter une commande dans un terminal Bash, le shell parent duplique sa structure via l'appel système fork(), puis le fils utilise exec() pour écraser la copie par les nouvelles instructions binaire à exécuter.

- • Programme vs Processus : Un programme est un fichier binaire exécutable passif résidant sur le disque. Un processus est l'instance active en cours d'exécution chargée en mémoire RAM avec des ressources attribuées (CPU, mémoire, descripteurs de fichiers).
- • Multitâche Préemptif & Ordonnanceur : Le noyau (kernel) découpe le temps processeur en tranches ultra-courtes appelées quanta. L'ordonnanceur (scheduler) attribue ou retire la main du CPU aux différents processus pour simuler une exécution parallèle.
- • Process Control Block (PCB) : Structure de données interne au noyau qui sauvegarde l'état complet d'un processus : état CPU, pointeurs mémoire (pile, tas, code), identifiants (PID, PPID, UID), priorité et descripteurs de fichiers ouverts.
- • Arborescence et Filiation (PID/PPID) : Tout processus possède un identifiant unique (PID - Process ID) et un identifiant parent (PPID - Parent Process ID). Ils forment un arbre dont la racine absolue est systemd ou init (PID 1).
- • Mouvement de Création fork() et exec() : L'appel système fork() clone le processus parent à l'identique en lui donnant un nouveau PID. L'enfant exécute ensuite exec() pour remplacer sa carte mémoire par le code du nouveau programme.
- • Modèle des 3 États : Élu (exécute sur le CPU), Prêt (possède ses ressources RAM et attend du temps CPU), Bloqué (suspendu en attente d'une opération d'E/S ou d'un événement externe). Un processus Bloqué repasse toujours par l'état Prêt avant de redevenir Élu.

Cas concret enseignant : *L'analogie du livre de recettes : Le professeur compare le programme sur disque à un livre de recettes rangé sur une étagère de bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter la recette dans sa cuisine (dynamique, occupant de la place en RAM sur le plan de travail, consommant du temps CPU lors de la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme la plus élémentaire de communication inter-processus (IPC). Lorsqu'un signal atteint un processus cible, son fil d'exécution normal est instantanément interrompu pour réagir au signal. À la réception d'un signal, trois comportements sont possibles : exécuter l'action par défaut du système (souvent l'arrêt du processus), ignorer le signal (ignore), ou l'intercepter (catch) au moyen d'une routine logicielle nommée gestionnaire de signal (signal handler). Cela permet à des applications comme des éditeurs de texte de sauvegarder les fichiers temporaires lors d'une demande de fermeture. Néanmoins, la spécification POSIX préserve l'autorité absolue de l'administrateur système en définissant deux signaux d'exception ininterceptables : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces deux signaux sont interceptés et traités en direct par le noyau lui-même. Aucun code applicatif ne peut les détourner, garantissant ainsi la capacité d'arrêter n'importe quel processus verrouillé dans une boucle infinie ou malveillant.

- Signal POSIX : Notification asynchrone ou interruption logicielle transmise par le noyau à un processus pour l'avertir d'un événement système ou lui imposer une instruction.
- Gestionnaire de signal (Signal Handler) : Fonction personnalisée qu'un programmeur peut attacher à un signal pour intercepter (catch) le signal au lieu d'exécuter l'action système par défaut.
- SIGINT (Signal 2) : Émis via le raccourci 'Ctrl + C' dans le terminal. Demande une interruption douce d'un processus interactif.
- SIGTERM (Signal 15) : Signal de terminaison propre envoyé par défaut par la commande 'kill '. Il invite le processus à sauvegarder son état, fermer ses fichiers et sockets avant de s'éteindre.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel exécuté directement par le noyau Linux. Il détruit immédiatement le processus et ne peut être ni intercepté, ni masqué, ni ignoré.
- Job Control (SIGSTOP 19 / SIGCONT 18) : Raccourci 'Ctrl + Z' envoyant SIGSTOP pour suspendre l'exécution. Les commandes 'fg' (foreground) ou 'bg' (background) envoient SIGCONT pour reprendre l'exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos travaux et fermer la porte à clé). SIGKILL (9) est l'intervention de la brigade de sécurité qui enfonce la porte et vous dégage sur-le-champ sans vous laisser toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Dans la syntaxe du shell Bash, la commande kill attend la structure `kill - `. Le tiret `-` est obligatoire pour indiquer que le nombre 9 représente le numéro du signal (SIGKILL). Si vous omettez le tiret et tapez `kill 9 1234`, l'utilitaire `kill` interprète '9' non pas comme un signal, mais comme un premier PID à détruire, et essaie d'envoyer le signal 15 par défaut aux processus PID 9 et PID 1234. Si le PID 9 n'existe pas ou ne vous appartient pas, le système lève une erreur de syntaxe ou de permission.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie (marqué état 'Z' ou '') est un processus enfant qui a déjà terminé son exécution, mais dont le processus parent n'a pas encore lu le code de retour via l'appel système `wait()` ou `waitpid()`. Il n'occupe plus de mémoire RAM ni de CPU, mais conserve sa ligne dans la table des processus du noyau. On NE PEUT PAS tuer un processus Zombie avec la commande `kill` (même avec `kill -9`) car il est déjà mort ! Pour le faire disparaître de la table, il faut soit forcer le père à exécuter son `wait()`, soit détruire directement le processus père (`kill -9 `). Une fois le père détruit, le processus Zombie est adopté par systemd/init (PID 1) qui nettoie immédiatement sa structure.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Expliquez le rôle exact du symbole '&', détaillez ce qui se passe lors de l'exécution de la commande kill -15 et décrivez le résultat de la vérification par ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' indique au shell d'exécuter la tâche en arrière-plan (background). Le terminal affiche l'identifiant de job (ex: [1]) ainsi que le PID attribué par le noyau (ex: PID 4521) et rend immédiatement la main à l'utilisateur sans bloquer l'invite de commande. Étape 2 : La commande 'ps -ef | grep sleep' parcourt la liste globale des processus système et extrait les lignes contenant le terme 'sleep'. Cela permet d'isoler la tâche lancée à l'étape 1 et de repérer son PID (4521) ainsi que son PPID (le PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. L'utilitaire sleep n'ayant pas de gestionnaire personnalisé modifiant ce comportement, il applique le comportement système par défaut : l'arrêt propre de la tâche et la libération de ses ressources.

Résultat : Le processus 'sleep 600' est détruit proprement par le signal SIGTERM. Une seconde exécution de 'ps -ef | grep sleep' confirme sa disparition définitive de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le tout premier processus utilisateur est systemd (ou init sur les systèmes anciens), et son PID est strictement 1.

Q : Quel signal POSIX est généré et transmis au processus au premier plan lorsqu'un utilisateur tape la combinaison 'Ctrl + C' dans son terminal ?

R : Le raccourci 'Ctrl + C' génère le signal SIGINT (Signal 2), qui demande l'interruption modérée de la tâche.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : SIGKILL ne peut pas être intercepté car il n'est jamais transmis au processus lui-même : il est réceptionné et exécuté directement par le noyau Linux pour offrir à l'administrateur un moyen garanti à 100% de neutraliser tout processus.